

УПРАВЛЕНИЕ ПАМЯТЬЮ

Управление памятью – отображение информации в память с помощью 3-х функций:

1. **Именуемая функция (И)** – однозначно отображает данное пользователем имя (**ПИ**) в идентификатор информации (программный адрес – **ПА**), к которой это имя относится (переменные, файлы и др. объекты).
2. **Функция памяти (П)** – отображает идентификаторы (программные адреса) в номера физических ячеек памяти (физические адреса – **ФА**), в которых они находятся.
3. **Функция содержимого/значения (З)** – отображает каждый адрес памяти в значение, которое находится по этому адресу.



Результат каждого отображения зависит от времени:

Именуемая – не устанавливается до тех пор, пока задание не будет связано с системными модулями и файлами, которые используются в задании.

Памяти – фиксируется после загрузки задания (как правило).

Значения – меняется всякий раз при выполнении команды записи в память. Эта функция полностью определяется алгоритмом программы (пользователем).

Отображения Именуемой функции и функции Памяти находятся в ведении системы.

При именовании данное пользователем имя может относиться к различным объектам в зависимости от контекста, в котором оно употребляется. Для ОС контекст это текст программы пользователя. Пользователи могут дать одни и те же имена различным объектам. ОС должна установить **соответствие каждому имени** в каждом контексте пользователя **нужного объекта**. Необходимо также, чтобы один объект мог именоваться разными именами в различных программах.

Структура программ и распределение памяти связаны и зависят друг от друга!

ОБЪЕДИНЕНИЕ МОДУЛЕЙ

Обработка программ включает этапы:

1. **Компиляция.** На вход подается *текст программы*, на выходе получаем *объектный код*.
2. **Сборка.** На вход подается *объектный код*, на выходе получаем *загрузочный код*.
3. **Загрузка.** На вход подается *загрузочный код*, на выходе получаем *рабочий код*.

Основная проблема этапов 2 и 3 – объединение модулей (разрешение внешних ссылок).

Процесс разрешения ссылок может быть выполнен 3-мя способами:

1. Объединение различных процедур во время компиляции. Реализуется в большинстве языков программирования.
2. Предварительно скомпилированные программы объединяются либо до (на этапе сборки) либо во время загрузки. Отдельный шаг между компиляцией и загрузкой.
3. Объединение откладывается до запуска на счет и осуществляется системой автоматически в процессе счета = динамическое разрешение ссылок. Данный способ

используется, когда имена некоторых процедур неизвестны до начала выполнения программы и характер вычислений зависит от входных данных.

СПОСОБЫ ОБЪЕДИНЕНИЯ МОДУЛЕЙ

Сборка как предварительный этап. Группа объектных модулей объединяется в 1 модуль в относительных адресах с началом в нулевой ячейке. Объединенный модуль – переместимый объект и при загрузке в ОЗУ абсолютный адрес первого слова помещается в **Базовый регистр**. Тогда абсолютный адрес это сумма содержимого базового регистра и относительного адреса в объединенном модуле. После редактирования задание – переместимый модуль загрузки, начинающийся с нулевого адреса.

Сборка при загрузке. Исходная информация для загрузчика, объединяющего модули (совокупность модулей + дополнительные данные):

1. Объектный модуль.
2. Длина модуля.
3. Имена описанные в модуле, на которые могут ссылаться другие модули (ссылки внутрь).
4. Имена не описанные в данном модуле, на которые есть ссылки в данном модуле (ссылки наружу).

Загрузчик, объединяющий модули, реализует объединение объектных модулей, настройку адресов, загрузку объединенной программы в ОЗУ.

Загрузчик составляет таблицу имен, на которые могут быть ссылки из других модулей (ссылки внутрь). В таблице каждому имени ставится в соответствие запись следующего содержания:

- а) Если имя описано в модуле, который уже загружен, то запись содержит адрес этого имени относительно начала объединенного имени.
- б) Если объектный модуль, в котором определено это имя, еще не загружен (адрес неизвестен), то запись содержит ссылку на связанный список адресных полей, в которых в дальнейшем нужно будет поместить адрес, соответствующий данному имени. Когда это имя встретится загрузчику, он просмотрит список и вставит адрес этого имени по всем указателям списка.

Все модули загружаются последовательно в ОЗУ. Относительные адреса сдвигаются на сумму длин уже загруженных модулей. Каждая внешняя ссылка либо разрешается (если модуль уже загружен), либо присоединяется к списку, соответствующему этому имени. После загрузки таблицу можно сохранить для отладки.

Узловые моменты привязки приложения к памяти:

1. Программирование в абсолютных адресах – при написании программы указываются абсолютные адреса (объединение отображений **И** и **П** раз и навсегда).
2. Программирование в символьных обозначениях – компилятор порождает адреса вместо символьных имен.

А) Порождаемые адреса – фиксированные абсолютные адреса (привязка **П** происходит сразу же за **И**).

Б) Порождаемые адреса – допускают настройку и при запуске получают абсолютные значения, не меняющиеся при выполнении программы (привязка к памяти **П** происходит во время загрузки).

В) Порождаемые адреса допускают настройку и получают абсолютные значения при каждом обращении к ним (привязки **П** как таковой нет = динамическое распределение).

Основной вопрос задачи распределения памяти состоит в том, как объединенный модуль записать в ОЗУ. Далее исходим из того, что задания переместимы.

Стратегии распределения памяти.

1. Перекрывание программ в памяти (планируется пользователем).
2. Попеременная загрузка заданий.
3. Сегментная организация программ.
4. Страничная организация памяти.
5. Сочетание сегментной организации программ со страничной организацией памяти.

ПЕРЕКРЫТИЕ ПРОГРАММ

Условие перекрывания: 2 модуля могут располагаться в одних и тех же ячейках памяти, если: а) модулям не нужно присутствовать в ОЗУ одновременно, б) для сохранения межмодульной коммуникационной информации (обычно через поле общей памяти) принимаются необходимые предосторожности.

Для реализации перекрывания редактор связей порождает специальные команды, вызывающие загрузчик всякий раз, когда происходит обращение к одной из перекрывающихся программ (загрузчик вводит модуль в ОЗУ, если он еще не загружен).

Проблемы:

Программист не всегда может априори задать описание перекрываний (т.к. может не знать нужно ли присутствовать перекрывающимся модулям в памяти одновременно).

Т.к. перекрывания планируются пользователем заранее, а степень загрузки памяти и поведение других программ предвидеть заранее невозможно, то эта стратегия не дает существенного улучшения использования всей памяти (особенно при низкой нагрузке на вычислитель).

ПОПЕРЕМЕННАЯ ЗАГРУЗКА ЗАДАНИЙ

ОС осуществляет загрузку/выгрузку заданий целиком после начальной загрузки в процессе выполнения смеси заданий. Стратегия позволяет избирательно загружать задания с целью увеличения загрузки всех компонент вычислителя. Основной недостаток – внешняя фрагментация (пустоты памяти между заданиями недостаточного размера для загрузки других заданий).

СЕГМЕНТНАЯ ОРГАНИЗАЦИЯ ПРОГРАММ

Сегментация программ – прием, при котором адресная структура программы отражает ее содержательное членение. Пространство адресов программы разделяется на отрезки – сегменты различной длины, соответствующие содержательно различным частям программы. Сегмент = процедура, область описания данных. При сегментации АДРЕС состоит из 2-х частей:

1. Имя сегмента **S** (его номер).
2. Адрес слова внутри сегмента **d** (смещение).

Реализация

Каждому заданию (приложению), состоящему из набора сегментов, соответствует ТАБЛИЦА СЕГМЕНТОВ (ТС), в которой для каждого сегмента имеется запись (строка). Адрес ТС хранится в аппаратном РЕГИСТРЕ ТАБЛИЦЫ СЕГМЕНТОВ (РТС) каждого процессора.

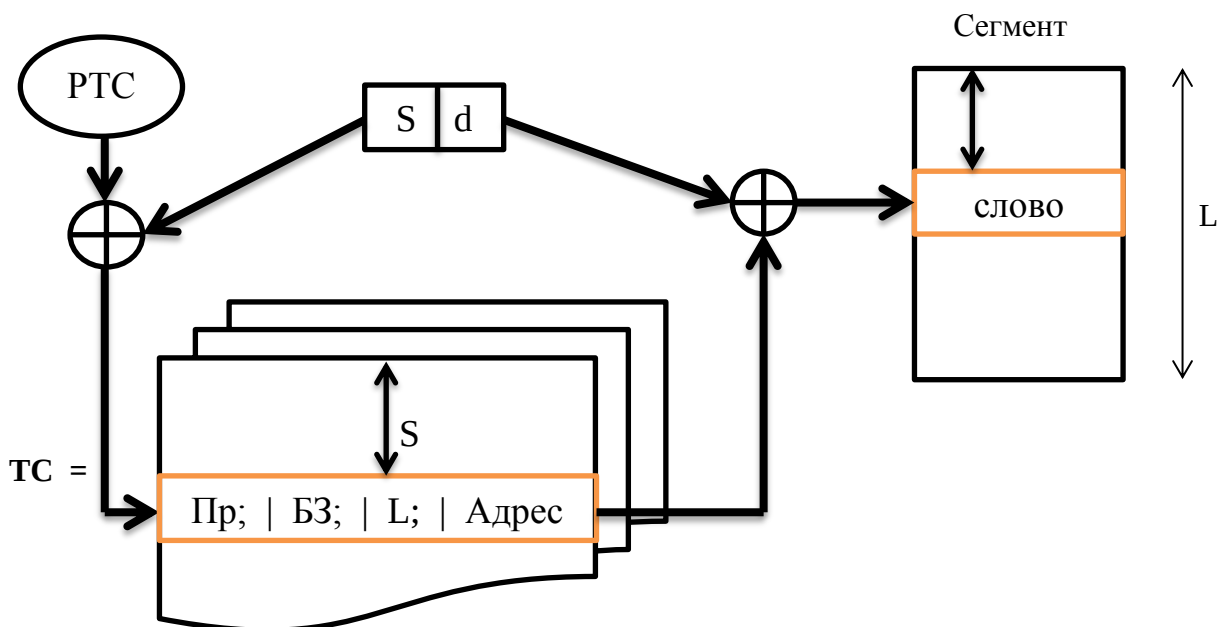
В S-й строке ТС (имя сегмента=№строки) находится:

1. Признак присутствия сегмента в ОЗУ (**Пр**).
2. Адрес (Базовый регистр) S-го сегмента (**Адрес**).
3. Длина (Граница) сегмента (**L**).
4. Биты защиты для контроля доступа к сегменту (**БЗ**).

Доступ к слову [S,d]:

1. По РТС обращение к ТС.
2. Проверяется присутствие сегмента в памяти (по признаку **Пр**)
3. При наличии сегмента в ОЗУ в S-й строке выбирается **Адрес** сегмента.
4. В d-й ячейке выбирается искомое слово.

Если сегмента в ОЗУ нет – происходит прерывание из-за отсутствия сегмента в памяти.



Переключение системы на другую программу (приложение) сводится к замене содержимого РТС на адрес другой таблицы.

- ⊕
- а) Сегменты могут размещаться в несмежных областях памяти.
 - б) Не все сегменты должны одновременно находиться в памяти (часть может располагаться на ВнУ).
 - в) Перекрытия реализуются системой без предварительных указаний программиста. Возможно перекрытие областей отдельных программ. При этом эффективно используется память.
 - г) Упрощается объединение модулей. Операция установления внешних связей сводится к просмотру таблицы для отыскания адреса (S,d), соответствующего внешней ссылке (не нужны эти функции в редакторе и загрузчике).
 - д) сегментация облегчает организацию разделения параллельно используемых программ. Для этого информацию о параллельно используемой программе вносят несколько таблиц сегментов. Сама программа при этом обрабатывает разные сегменты данных, описанные в таблицах различных приложений.

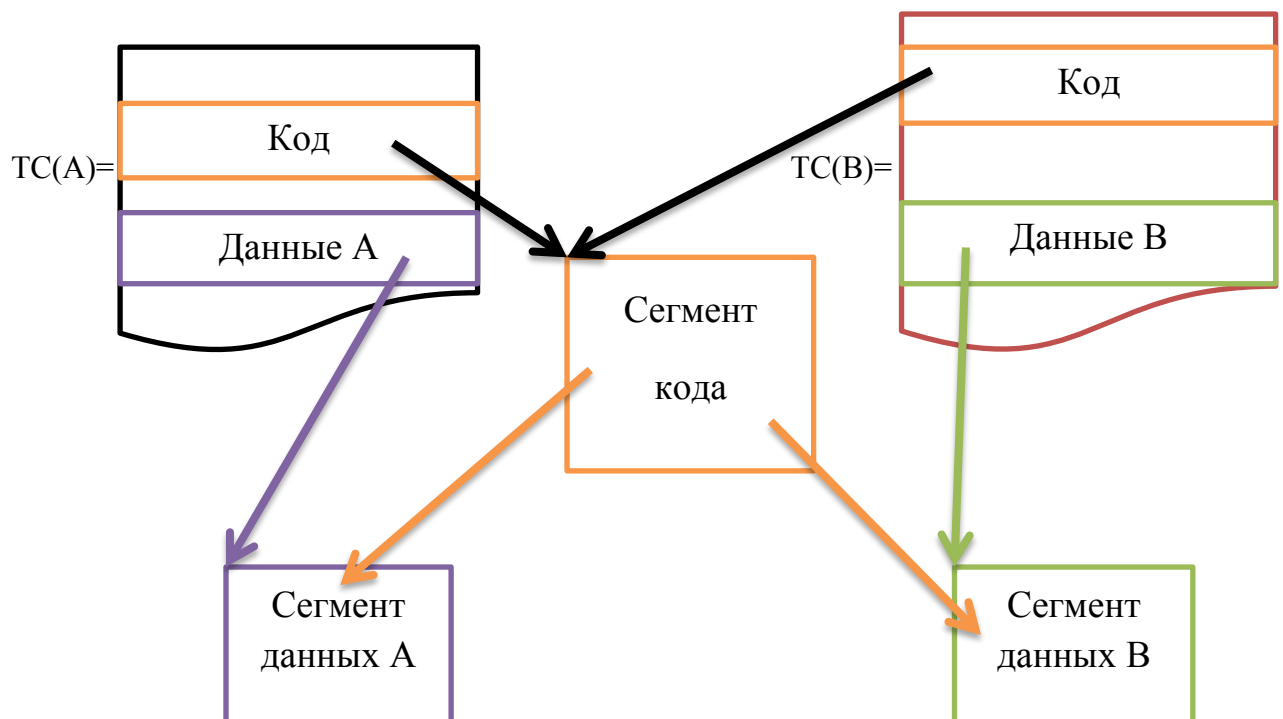
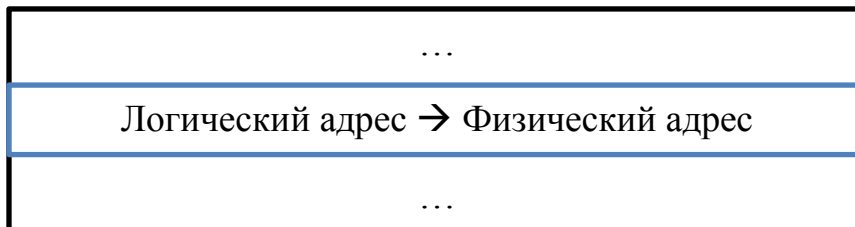


А) Внешняя фрагментация (пустоты между загруженными в память сегментами, размеры которых малы для размещения других сегментов – образуются из-за загрузки-выгрузки сегментов в процессе вычислений). Профилактика: страничная организация памяти!

Б) Высокие накладные расходы на сопровождение миграции сегментов между ОЗУ и внешней памятью. Профилактика: страничная организация памяти!

В) Два обращения к ОЗУ (первое – к таблице за адресом сегмента, второе – к адресуемому объекту внутри сегмента). Существенно снижает быстродействие вычислителя. Профилактика: полностью ассоциативное кэширование адресных ссылок на сегменты (емкость кэша адресных ссылок TLB обычно несколько сотен строк), реализуется с помощью отображения:

Вход=ID-задачи+S (логический адрес)→ Физический адрес сегмента=Выход



СТРАНИЧНАЯ ОРГАНИЗАЦИЯ ПАМЯТИ

Страничная организация памяти – способ управления памятью, при котором пространство адресов памяти разбивается на блоки фиксированной длины – страницы (обычно размер = степень двойки).

Память – набор страниц равной длины.

Программы и данные – делятся на страницы того же размера, что и физические страницы ОЗУ, при этом программа/данные занимает целое число страниц, последняя страница может быть заполнена не полностью. Страница информации загружается в физическую страницу памяти.

Адрес: пара $[P, d]$, P – имя (номер) страницы, d – смещение. Задание – таблица страниц (ТСтр) содержит список страниц, составляющих это задание. Запись в таблице соответствует странице и содержит:

1. Признак наличия страницы в памяти.
2. Указатель местоположения страницы (Адрес страницы).
3. Биты защиты для контроля способа доступа.

Регистр таблицы страниц (РТСтр) центрального процессора указывает местоположение таблицы страниц текущего (выполняемого в данный момент) задания.

Доступ к объекту по логическому адресу $[P, d]$:

1. По регистру РТСтр выполняется обращение к ТСтр.
2. В P -й записи (строке) ТСтр выбирается адрес страницы.
3. В d -й ячейке страницы находим искомое слово.

Если требуемой страницы нет в ОЗУ, то происходит прерывание из-за отсутствия страницы.

 **Плюсы** – те же, что и при сегментной организации программ, **кроме** возможности организации совместного параллельного использования программ. **Дополнительно:** в отличие от сегментной организации программ при страничной организации памяти существенно легче реализуется миграция страниц на различные уровни подсистемы памяти.



А) Два обращения к ОЗУ. Профилактика – кэширование отображения логических адресов страниц в физические (решается, так же как и в случае сегментной организации программ).

Б) Внутренняя фрагментация. Пустоты возникают внутри последней страницы приложения. Возникает задача поиска компромисса между: издержками, связанными с движением страниц (желательно размер страницы увеличивать), и издержками внутренней фрагментации (желательно размер страницы уменьшать).

СОЧЕТАНИЕ СЕГМЕНТНОЙ ОРГАНИЗАЦИИ ПРОГРАММ И СТРАНИЧНОЙ ОРГАНИЗАЦИИ ПАМЯТИ

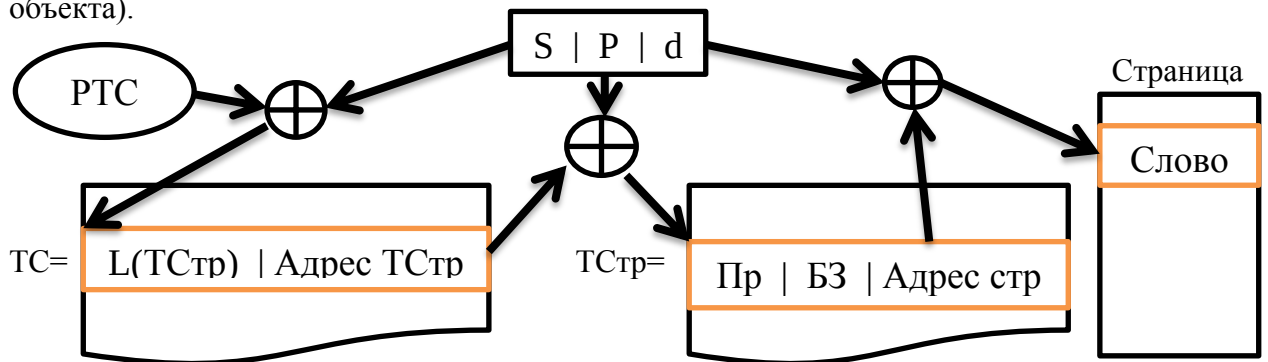
Память вычислителя разбивается на страницы фиксированной длины.

Программа – набор сегментов.

Сегмент – набор страниц.

Адрес – тройка: $[S, P, d]$, S – номер сегмента приложения, P – номер станицы данного сегмента S , d – смещение внутри страницы P .

Для выбора адресуемого объекта при такой трехкомпонентной ссылке нужно 3 цикла обращения к памяти (в таблицу сегментов за адресом таблицы страниц, в таблицу страниц за адресом страницы с адресуемым объектом, в страницу за значением адресуемого объекта).



$\oplus$ Сегментно-страничному способу распределения памяти присущи все преимущества сегментной организации программ и страничной организации памяти (сегментация позволяет просто организовать совместное использование кода программ и установление внешних связей, страничная организация обеспечивает эффективное распределение памяти и исключает внешнюю фрагментацию).



Такая высокая способность приспосабливаться к различным требованиям достигается ценой 3-х компонентной адресной ссылки (3-х обращений к ОЗУ). Профилактика – кэширование отображения:

Вход=ID-задачи+S+P (лог. адрес) $\rightarrow$ **Физический адрес страницы**=**Выход**

ВИРТУАЛЬНАЯ ПАМЯТЬ

В системе с ВП у каждого процесса создается иллюзия. Что все приложение находится в ОЗУ. При обращении процесса к информации, которой нет в ОЗУ, вырабатывается прерывание, управление передается супервизору, который перемещает нужную информацию с внешнего устройства в ОЗУ и процесс получает доступ к ней.

В самом общем случае ВП организуется в виде многоуровневой памяти. Каждый уровень $M(i)$ имеет более высокую скорость доступа, меньшую емкость и более высокую удельную стоимость хранения по сравнению с уровнем $M(j)$ при $i < j$. Как правило, многоуровневая память разбита на страницы фиксированного размера. Объект, который находится в памяти i -го уровня, хранится также на уровнях с большими номерами. Если ЦП обращается к объекту, которого нет в памяти 1-го уровня, система ищет его на уровнях с большими номерами. Когда объект найден, он перемещается на верхние уровни вплоть до 1-го. При переносе объекта с более низкого уровня на более высокий (более быстрый) вытесняется какой либо объект – согласно стратегии вытеснения.

СТРАТЕГИИ РАСПРЕДЕЛЕНИЯ РЕСУРСОВ ПРИ СЕГМЕНТАЦИИ И СТРАНИЧНОЙ ОРГАНИЗАЦИИ.

Сегментация и страничная организация – это логический и физический подходы применяемые к пространству программных адресов и пространству адресов памяти.

Существенное различие между разбиением на сегменты и страницы состоит в том, что сегменты – блоки переменной длины, страницы – блоки фиксированной длины. Отсюда проблемы: размещения логических сегментов в памяти, разбиение программ на страницы.

Важнейшие вопросы реализации:

1. Как распределять память каждого уровня (для страничной организации задача проста – ОС ведет таблицу программных страниц и координат соответствующих физических страниц, для сегментации задача более сложная).
2. Какие критерии применять при подкачке блоков (различают 2 стратегии: опережающая и по запросу).
3. Какие критерии применять при вытеснении блоков (существует много стратегий).

1. СТРАТЕГИИ РАСПРЕДЕЛЕНИЯ ДЛЯ СЕГМЕНТОВ ПЕРЕМЕННОЙ ДЛИНЫ

Задача: найти стратегию, которая хорошо загружает память при минимальных издержках распределяющих алгоритмов.

Решение: список свободной памяти или уплотнение.

ВАРИАНТЫ СПИСКОВ СВОБОДНОЙ ПАМЯТИ.

Список свободной памяти, упорядоченный по возрастанию адресов.

Каждая пустота, входящая в список, содержит указатель на следующую и поле длины (количество блоков пустой области). Освобожденная область вставляется в список с сохранением упорядоченности адресов по возрастанию. Если предшествующая и (или)

последующая область списка пустот примыкают к вновь освобожденной, то они объединяются в одну пустоту.

Из списка пустот память может распределяться по 2-м стратегиям:

1. Выбор 1-й подходящей пустоты (при запросе пустоты размера k выделяется первая пустота размера $\geq k$).
2. Выбор самой подходящей пустоты (просматривается весь список и выбирается наименьшая пустота размера $\geq k$).

Если нет пустоты размера $\geq k$, то запрос остается неудовлетворенным, либо уплотнение (если сумма длин пустот в списке $\geq k$).

Во многих случаях 1-я стратегия лучше заполняет память, чем 2-я.

Список свободной памяти, упорядоченный по возрастанию размера пустот.

Поиск первой подходящей пустоты дает самую подходящую.

Модификация:

Список пустот дополняется списком указателей $\{P_1, P_2, \dots, P_m\}$, P_i – указывает на первую пустоту в исходном списке размера от iC до $(i+1)C$ блоков ($C = \text{const}$). Этот набор указателей позволяет ускорить поиск пустоты нужного размера, но при высокой динамике содержания списка его сопровождение очень дорогое. Главная трудность состоит в сложности вставки вновь освободившейся области в список пустот (требует много времени).

Организация пустот системой расщепления

Память вычислителя представляется блоками длины 2^k , $k \geq 0$.

Список k содержит блоки 2^k .

Пусть объем распределяемой памяти $= 2^s$, тогда в системе всего $S+1$ список $0 \leq k \leq s$. Сначала все списки пусты, кроме Списка S , который указывает на первое слово в распределяемой памяти.

При поступлении запроса на пустоту размера 2^k :

1. Просматривается Список k , если он не пуст, то выделяется пустота иначе...
2. Просматривается Список $k+1$, если он не пуст, один из его блоков расщепляется на 2 половинки, одна выделяется по запросу, а другая помещается в Список k , иначе...
3. Процесс поиска пустоты продолжается до тех пор, пока не будет найден непустой Список.
4. Если Списки J для $k \leq J \leq s$, то запрос не удовлетворяется.

При таком способе организации управления списками адрес каждого блока длины 2^k без остатка делится на 2^k , тогда для каждого блока размера 2^k легко найти адрес того блока длины 2^{k+1} , от которого он был отщеплен. Как только два парных блока длины 2^k – двойняшки, получившиеся в результате расщепления одного блока длины 2^{k+1} окажутся свободными, они объединяются.

УПЛОТНЕНИЕ

Если запрошена область длины превосходящей наибольшую пустоту, то возможны следующие действия:

1. Отказ в удовлетворении запроса.
2. Вытеснение какого-либо сегмента.
3. Уплотнение – перегруппировка памяти и объединение мелких пустот.

Обычно уплотнение выгодно когда: запросы на память велики по размеру и из-за внешней фрагментации образуется большой процент пустот, ЦП не занят выполнением активных заданий (простаивает) и его можно занять уплотнением – операция будет бесплатной.

2. СТРАТЕГИИ ПОДКАЧКИ ОБЪЕКТОВ

2 стратегии: **по запросу** (по прерыванию), **опережающая**.

Экономия опережающей подкачки определяется исключением прерываний (их число определяется объемом опережения) = затраты на обслуживание прерываний – затраты на опережающий выбор объектов. Этот выигрыш имеет место при условии. Что объекты будут введены не преждевременно и к ним будет обращение до вытеснения (при высокой вероятности использования объектов)! Обычно это наблюдается при известной логической связи между объектами (например – медиа файл). В большинстве случаев подкачка по запросу выгоднее опережающей подкачки.

3. СТРАТЕГИИ ВЫТЕСНЕНИЯ ОБЪЕКТОВ ИЗ ПАМЯТИ

Правила вытеснения оцениваются и сопоставляются по критерию того насколько хорошо они сокращают ожидаемый объем движения объектов между уровнями памяти. Нужно не вытеснять **НУЖНЫЕ** объекты!

Стратегии вытеснения:

1. **Случайный выбор** – легко реализуемая стратегия, но часто вытесняются **НУЖНЫЕ** объекты.
2. **FIFO** – первым пришёл – первым ушёл. Легко реализуется для страниц фиксированной длины – физические страницы образуют список из K элементов, упорядоченный по номерам страниц, указатель ссылается на самую новую страницу. Когда необходимо ввести какую-либо страницу содержимое указателя складывается по модулю K с единицей и страница, на которую указывает полученное значение, удаляется. Однако данная стратегия существенно ограничивает производительность системы т.к. самая старая страница не обязательно наименее нужная.
3. **По давности использования (LRU – Least Recently Used)**. Удаляется тот объект, обращения к которому не было дольше, чем к все остальные объекты. Трудно реализуемая дорогая стратегия (требует мониторинга частоты обращения), но в большинстве случаев работает хорошо!
4. **Предписанные пользователем (программистом) приоритеты**. Используется при известной связи приоритетов с **НУЖНОСТЬЮ** объектов.
5. **Приоритеты, задаваемые системой**.
6. **Оптимальное вытеснение**. Вытесняется тот объект, обращение к которому не произойдет дольше всего. Стратегия не реализуема т.к. в любой момент времени нельзя знать о будущих обращениях к памяти, **НО** используется как эталон для реализуемых стратегий при апостериорном анализе свойств реализуемой стратегии.
7. **Смешанные методы** – основаны на здравом смысле. Для вытеснения предлагается на выбор список возможностей, упорядоченных по убыванию степени их желательности. Например:
 - а) принадлежащий неактивной задаче объект, использованный только для чтения;
 - б) принадлежащий неактивной задаче объект, использованный только для записи;
 - в) объект управляющей программы, к которому не было обращения заданное время;
 - г) объект задачи, ожидающей в/в;
 - д) ...

Система ведет списки для каждой подгруппы. Когда необходимо вытеснить объект, он выбирается из наиболее желательного непустого списка.